

EE/ECE Coursework Report: TNU PIC18F Microcontroller Board Replication

Student Name: Jap Ja Mun Pan (Ah-Pan)

GitHub Username: Moonaround

Date: September 24, 2026

Phase 1: Researching GitHub for Hardware with AI

1-a) The Core Git-KiCad Query

The following prompt was submitted to analyze the baseline architectural shift from software tracking to hardware tracking:

"I am an engineering student with baseline GitHub experience managing software repositories. However, I have not used Git to manage an ECAD workflow before. What are the core architectural differences in how Git tracks KiCad files versus standard text files, and what specialized skills do I need to learn to safely version control a hardware layout?"

Result - Core Architectural Differences

The primary shift when moving from software version control to an ECAD workflow lies in how data is structured and modified. Standard software repositories contain deterministic, line-by-line source code where text changes correlate directly to functional changes. KiCad files (.kicad_sch for schematics and .kicad_pcb for board layouts) are formatted as **S-expressions** (highly nested, parenthesized text strings).

While they are technically text files, Git's default line-by-line diff and auto-merge engines do not understand the underlying spatial geometric relationships, netlist connections, or schematic coordinate matrices. A tiny visual shift on your canvas—such as moving a component by a fraction of a millimeter—can trigger hundreds of lines of text changes in the file, reordering the S-expressions completely. If two users make minor spatial adjustments simultaneously, Git's native auto-merge feature will frequently scramble the file syntax, resulting in **unreadable board layouts or corrupted netlists**.

Specialized Git Skills Needed for KiCad

To maintain project file integrity and safely collaborate on a hardware design repo, you must develop specific configuration habits and workflows:

1. Isolation of Local Workspace Metadata (.gitignore)

KiCad continuously creates transient locking profiles and local state databases while a project is open. If these are committed, they overwrite other users' display setups and cause immediate, non-functional merge conflicts. Your project requires a strict hardware-specific .gitignore file to isolate these elements:

- ***.bak and *-bak:** Local automatic schema and layout file backups.
- ***.lck:** Session locking markers that prevent multi-instance editing.
- **.kicad_prl:** Project local settings storing user-specific zoom states, panel locations, and cursor coordinates.

2. Atomic S-Expression Commits

In software development, bundling minor logical updates into a single large pull request is a common practice. In ECAD version control, you must adopt an **atomic commit structure**. Commits must be highly isolated, incremental milestones tied strictly to physical layout phases (e.g., *"Completed placement of decoupling capacitors for MCU"* or *"Routed +5V power rail matrix"*). This ensures that if an auto-merge corrupts the S-expression structural tree, you have an explicit historical path to roll back to without losing massive chunks of layout progress.

3. Visual Layout Verification Over Text Diffs

Standard text diff blocks (git diff) are virtually useless for interpreting graphical modifications like trace routing adjustments or ground plane pours. You must learn to rely on visual comparison pipelines rather than text-based tracking.

For advanced hardware engineering repositories, this involves setting up external graphical diffing frameworks (such as **KiCad-Diff**) locally, or utilizing web-based hardware pipeline renderers within your GitHub Pull Requests. These specialized tools generate graphical overlay maps—rendering modifications as color-coded trace layers—allowing you to visually review circuit shifts before merging them into your stable production branch.

1-b) Advanced Hardware Collaboration Strategy Query

The following prompt was submitted to analyze concurrent design tracking within multi-user engineering teams:

"I want to use GitHub to store a KiCad PCB project. What additional directions or best practices would be most useful for sharing binary schematic and layout files between multiple users of a KiCad project stored in GitHub to prevent file corruption or merge conflicts?" [1]

Result - When sharing a KiCad project on GitHub, managing collaboration requires moving away from traditional software development habits. Because KiCad schematics (.kicad_sch) and PCB layouts (.kicad_pcb) use deeply nested text blocks to represent physical geometries, simultaneous edits to the same file will almost certainly corrupt your netlists or circuit layers.

To collaborate smoothly with your professors or classmates without breaking your design files, implement these industry-standard hardware version control strategies:

1. Strict Modular Branching (The "One File, One Designer" Rule)

The most effective way to prevent merge conflicts is to avoid them entirely. Git cannot line-merge spatial layout changes automatically.

- **Establish a File Lock System:** Communicate explicitly with your team before pushing changes. Only one person should actively modify the PCB layout file (.kicad_pcb) or a specific schematic sheet (.kicad_sch) at any given time.
- **Utilize Hierarchical Sheets for Collaboration:** If multiple people must work on the schematic simultaneously, split the design into **hierarchical sub-sheets** within KiCad. This isolates functional blocks (e.g., one designer works on power_supply.kicad_sch while another works on microcontroller.kicad_sch). Because these are tracked as entirely separate files in Git, you can work concurrently without touching the same data.

2. Lock Down Your Hardware .gitignore

KiCad creates dynamic workspace caches and lockouts while you work. If multiple users push these background files to GitHub, they will continuously overwrite each other's desktop window configurations, resulting in tracking errors.

Ensure your .gitignore file contains these specific hardware exclusions:

text

Exclude auto-generated backups and session locks

*.bak

*-bak

*.lck

*.tmp

Exclude local user interface preferences (zoom, open panels, tab layout)

.kicad_prl

Optional: Exclude local manufacturing outputs if generated dynamically
fp-info-cache

Use code with caution.

3. Move from Text Diffs to Visual Diff Tracking

Reviewing a code change on GitHub by looking at a text diff is standard practice. Reviewing a PCB change by reading raw coordinate strings is impossible.

- **Integrate Graphical Review Tools:** Before merging a pull request, use open-source hardware diffing frameworks like **KiCad-Diff** or web-based image comparison pipelines. These tools render your schematic or PCB layers visually and overlay changes in contrasting colors (e.g., showing old traces in red and newly routed traces in green).
- **Mandatory Image Attachments on PRs:** Establish a team habit where every Pull Request must include a screenshot or 3D render of the changes made on the board canvas. This allows reviewers to verify the physical alterations instantly before clicking merge.

4. Commit as Atomic Layout Milestones

Do not wait until the entire board is finished to push to GitHub. If an unpredictable text distortion corrupts your file structure late in the project, tracing the error becomes a nightmare.

Adopt an **atomic commit strategy** mapped directly to your hardware progression checklist:

- *Commit 1:* " Staged micro-controller symbol and basic power bus pins."
- *Commit 2:* " Placed decoupling capacitors adjacent to MCU VDD paths."
- *Commit 3:* " Positioned external oscillator matrix and status indicators."

By making tightly isolated commits, you preserve clean rollback milestones. If a file breaks, you can revert to the exact point in time right before the corruption occurred without losing hours of manual alignment work.

Phase 2: Documenting Your Engineering Learning Plan

2-a) AI Responses for Technical Sourcing

Response to Core Git-KiCad Query (1-a):

Unlike traditional software workflows where Git tracks deterministic, line-by-line source code, KiCad projects rely on a collection of tightly coupled files. Starting in KiCad 6, 7, and

8, schematic files (.kicad_sch) and PCB layouts (.kicad_pcb) are structured as plain-text files using **S-expressions** (nested parentheses). While this makes them text-readable, Git's default auto-merge algorithms do not understand the underlying spatial geometric relationships or netlist connections of a printed circuit board.

To safely manage a KiCad layout using Git, an engineer must master:

- **Atomic Commits for Hardware:** Commits should be highly granular and atomic. Instead of staging massive changes, commit distinct physical milestones (e.g., *"Completed placement of decoupling capacitors for MCU"* or *"Routed +5V power rail matrix"*). This provides a clean historical rollback trail if a file gets corrupted.
- **Project File Isolation:** KiCad tracks local UI states (zoom levels, open panels, tab selections) in auxiliary files like .kicad_prl (Project Local settings). Tracking these causes constant, meaningless merge conflicts between different computers. They must be actively ignored using configuration blockers.

Response to Hardware Git Strategy Query (1-b):

When multiple layout designers push modifications to a single central repository simultaneously, merge conflicts in an S-expression file can completely break the schematic netlist or PCB copper geometries. To successfully collaborate on a shared KiCad board design without file corruption, use these industry-standard best practices:

1. **Strict Modular Branching (No Shared Files):** Never allow two designers to modify the same schematic sheet or board layout file concurrently. Divide work explicitly by sheets or functional modules (e.g., User A works on the power regulation sheet, while User B builds the microcontroller interface pins on a separate sheet).
2. **Establish a Robust Hardware .gitignore:** Block all non-essential transient metadata. KiCad dynamically generates locking files (.lck) and auto-backup footprints (-bak) during live design sessions. If these are pushed to the cloud, they overwrite other users' environments.
3. **Utilize Visual Diff Tracking:** Text-based git diff tools (git diff) are unreadable for complex PCB matrices. Teams should use open-source hardware diffing frameworks like **KiCad-Diff** or web-based layout preview tools. These render the graphical sheets side-by-side or overlay copper changes as a visual color map during the Pull Request review phase.

Phase 2: Documenting Your Engineering Learning Plan

2-b) Selected Tutorial

- **Path Chosen:** Video Masterclass: *Version control for hardware developers // Git, GitHub, KiCad* by **Hutscape**.

<https://www.youtube.com/watch?v=V8yYrUN0q-0&t=131s>



```
* 4f0ece4 - (23 hours ago on 2021-01-28) feat: integrate lora and e-ink in PC
* 934f01b - (23 hours ago on 2021-01-28) feat: add password secret by Sayanee
* 4eecf64 - (23 hours ago on 2021-01-28) Merge pull request #2 from sayanee
| \
| * 21cafae - (23 hours ago on 2021-01-28) refactor: abstract lora and e-ink
| * e273169 - (23 hours ago on 2021-01-28) feat: add e-ink in firmware by Sayanee
| /
* 1b32002 - (23 hours ago on 2021-01-28) feat: add e-ink in firmware by Sayanee
* 05dadcc - (23 hours ago on 2021-01-28) feat: add e-ink in firmware by Sayanee
| \
| * a3863b3 - (24 hours ago on 2021-01-28) feat: add e-ink in firmware by Sayanee
| * bef0448 - (24 hours ago on 2021-01-28) feat: add firmware for the Lora controller
| /
* d2babab - (24 hours ago on 2021-01-28) chore: update git submodule for kicad
* 772a37c - (24 hours ago on 2021-01-28) chore: add library as a submodule by Sayanee
* 5d0a6e1 - (24 hours ago on 2021-01-28) fix: remove BATT net label and open
* 3c7c222 - (24 hours ago on 2021-01-28) docs: add readme by Sayanee
* 9bd98a4 - (24 hours ago on 2021-01-28) first commit by Sayanee
```

2-c) Justification Statement

"I chose this specialized hardware tutorial because it avoids dry software-centric examples and focuses directly on how Git handles KiCad schematics and layout databases. Watching a visual, step-by-step breakdown of managing atomic hardware commits, configuring a true electronics-focused .gitignore template, and structuring pull requests for circuit revisions matches my active engineering workflow."

2-d) Action Phase

To gain hands-on experience, the local Ubuntu terminal was used to navigate into the active project workspace directory, initialize a tracking repository from scratch to resolve missing metadata faults, define configuration rules, and execute an initial atomic deployment sequence to GitHub:

1. **Workspace Navigation & Repository Initialization:** Open the terminal window, enter the project directory path containing the asset files, and initialize the tracking engine to establish a hidden .git folder structure:

```
cd /home/jap/Desktop/junior_capstone
mv git_github_kicad_project1 TNU_PIC18F_Microcontroller_Board
cd TNU_PIC18F_Microcontroller_Board
git init
```

Use code with caution.

2. **Status Verification & Reviewing Unstaged Assets:** Execute a status check to verify the folder has shifted out of a fatal fault state and into an active tracking state, displaying the reference specifications as untracked files:

```
bash
git status
```

Use code with caution.

3. **Configuring a Hardware-Specific Exclusion Rules File:** Generate a custom .gitignore text configuration directly via the terminal stream to prevent transient local UI caches and automatic layout text backups from muddying repository logs:

```
bash
cat <<EOF > .gitignore
*.bak
*.lck
*~bak
.*
.kicad_prl
EOF
```

Use code with caution.

4. **Executing the First Atomic Hardware Commit:** Stage all components inside the directory path structure and commit the snapshot state with a precise descriptive logging label:

```
bash
git add .
git commit -m "feat: establish project environment and hardware-specific tracking rules"
```

Use code with caution.

5. **Linking and Deploying to the Remote Cloud Profile:** Establish an upstream association bridge link to the active repository layout template and broadcast the local progress securely to GitHub:

```
bash
```

```
git remote add origin https://github.com  
git branch -M main  
git push -u origin main
```

Use code with caution.

2-e) Proof of Progress

(PASTE YOUR TERMINAL SECTOR DISPLAYING THE SUCCESSFUL PUSH TRANSMISSION STREAM OR REPOSITORY LANDING TIMELINE SCREENSHOT DIRECTLY BELOW)

Phase 3: Repository Setup

3-a) Creating the Remote Repo

1. Log into your GitHub account online at ://github.com.
2. Click the green **New** button to initialize a brand-new cloud repository layout.
3. Set the **Repository Name** exactly to: TNU_PIC18F_Microcontroller_Board [1]
4. Set the visibility parameters to **Public** so project collaborators can access the branch history.
5. Leave the "Add a README file" checkbox **unchecked** to avoid any structural file merging conflicts during the initial terminal push sequence.
6. Click **Create repository**.

Phase 4: Professor Collaboration Invitation

4-a) Adding Your Instructors

1. Navigated to the repository settings menu of the new TNU_PIC18F_Microcontroller_Board page.
2. On the left-hand sidebar menu, clicked on **Collaborators**.
3. Clicked the green **Add people** button.
4. Typed in the professors' user handles carefully: **mdnardin** and **fcawth**.
5. Clicked **Add to this repository** to dispatch the contributor invitations.

Collaborators and teams

 **Public repository**
This repository is public and visible to anyone Manage visibility

Direct access 
2 entities have access to this repository. [0 collaborators](#). [2 invitations](#).

Manage access Add people

☐ Select all

Type ▾

☐	 Fred Cawthorne Awaiting fcawth's response	Pending Invite 	
☐	 mdnardin Awaiting mdnardin's response	Pending Invite 	

[< Previous](#) [Next >](#)

Phase 5: Initializing Your Local KiCad Workspace

Using the local terminal on Ubuntu, the following commands were executed to establish a localized working repo and isolate unnecessary hardware configurations:

5-a) Create Your Local Project Folder

```
bash
```

```
cd /home/jap/Desktop/junior_capstone/TNU_PIC18F_Microcontroller_Board
```

Use code with caution.

5-b) Initialize Git Tracking & Configure Your KiCad .gitignore

Initialize the hidden tracking environment:

```
bash
```

```
git init
```

Use code with caution.

Next, a dedicated .gitignore file was compiled to prevent local interface caches and backups from muddying repository branch tracking. Built instantly using:

bash

```
cat <<EOF > .gitignore
# KiCad backup and lock files
*.bak
*.lck
*~bak
.*~
```

```
# Workspace local settings cache
.kicad_prl
EOF
```

Use code with caution.

5-c) Link Your Local Directory to Your Moonaround Profile

bash

```
git remote add origin https://github.com
```

Use code with caution.

Phase 6: KiCad Part Placement & BOM Matching

6-a) Project Environment Creation

- Opened the KiCad development environment on Ubuntu.
- Selected **File -> New Project** from the tool menu tree.
- Named the layout file system exactly: TNU_PIC18F_Microcontroller_Board.
- Directed the configuration files to save directly inside the local initialized tracking root folder path: /home/jap/Desktop/junior_capstone/TNU_PIC18F_Microcontroller_Board/
- Opened the Schematic Editor (.kicad_sch file).

6-b) Component Sourcing & Mapping Strategy

The part sourcing phase was managed by cross-referencing two distinct engineering documents side-by-side with the KiCad workspace layout grid:

- **Cadence_Pic40_TNU_PCB_schematic_v2.pdf**: Relied upon to map visual arrangement patterns and circuit block topography across the sheet grid layout.

- **Cadence_Pic40_TNU_Bill_Of_Material.xlsx:** Relied upon to source exact manufacturing values, precise electrical properties, and footprint designations to refine generic symbol placements.

6-c) Step-by-Step Schematic Part Placement

- **Place the Microcontroller Brain:** Hovered the cursor over the center grid area and pressed the A hotkey to launch the Add Symbol selector catalog. Searched for the 40-pin microcontroller block (the primary PIC18F variant specified on row 1 of the ledger) and anchored it centrally.
- **Populate the Passive Networks:** Used the A hotkey interface recursively to fetch standard passive component layers:
 - Typed R to place current-limiting and pull-up resistor structures near core system rails.
 - Typed C to isolate unpolarized ceramic filtering components, dropping them parallel to the **VDD** and **VSS** pins to suppress high-frequency line noise.
 - Typed C_Polarized to map tantalum bulk power isolation tank nodes across the supply line paths.
- **Establish External Interface Layout Blocks:** Sourced and staged additional hardware modules across the canvas border perimeters—incorporating clock oscillators, reset buttons, status LEDs, barrel power inputs, and breakout headers matching the Cadence map architecture.
- **Parametric Property Configuration Updates:** Double-clicked each placed symbol to call up its Symbol Properties control panel. Upgraded the default generic fields to match physical engineering values from the spreadsheet (e.g., swapping standard fields to 10k for reset lines, 470 for current-throttling LED lines, and 0.1uF for noise bypassing).

6-d) Baseline Progress Visualization

